

Table des matières

I. Introduction.....	2
II. Transformation de Tseitin.....	2
1. Idée de la méthode.....	2
Exemples.....	2
2. Implantation de la fonction Tseitin.....	3
3. Implantation de la fonction restric_inter.....	3
4. Implantation de la fonction extention_inter.....	3
III. Génération aléatoire d'une formule.....	4
IV. Implémentation : test empirique de validité des résultats.....	4
V. Évaluation des performances.....	6
VI. Conclusion.....	6

I. Introduction

La transformation de Tseitin est une méthode essentielle en informatique théorique permettant d'éviter l'explosion combinatoire lors de la mise en forme clausale conjonctive de formules logiques. Dans ce rapport nous allons voir l'implémentation de cet algorithme en OCaml et les méthodes de test empirique pour l'évaluation des résultats obtenues.

II. Transformation de Tseitin

1. Idée de la méthode

La transformation de Tseitin consiste à ajouter de nouveaux atomes représentant des sous-formules non atomiques de la formule initiale en incluant leurs définitions à l'aide de l'opérateur d'équivalence, ce qui va empêcher l'explosion combinatoire lors de la mise en forme conjonctive de formules logiques.

Exemples

Prenons la formule suivante comme exemple :

soit $f = \text{Ou}(\text{Non}(\text{Atome 'a'}), \text{Atome 'b'})$ une formule logique dont son format infixe est $(\sim a + b)$,

Soit tf la transformée de Tseitin de f donc

$tf = \text{Et}$

$(\text{Atome "new_0",}$
 $\text{Et}(\text{Equiv}(\text{Atome "new_0", Ou}(\text{Atome "new_1", Atome "old_b"})),$
 $),$
 $\text{Equiv}(\text{Atome "new_1", Non}(\text{Atome "old_a"})))$

)et son format infixe est $(\text{new_0} * ((\text{new_0} \leftrightarrow (\text{new_1} + \text{old_b})) * (\text{new_1} \leftrightarrow \sim \text{old_a})))$.

```
Type #utop_help for help about using utop.

-( 20:24:19 )-< command 0 >
utop # let f = Ou(Non (Atome "a"), Atome "b");;
val f : formule = Ou (Non (Atome "a"), Atome "b")
-( 20:24:20 )-< command 1 >
utop # string_of_formule f;;
- : string = "(~ a + b)"
-( 20:25:15 )-< command 2 >
utop # let tf = tseitin f;;
val tf : formule =
  Et
    (Atome "new_0",
      Et
        (Equiv (Atome "new_0", Ou (Atome "new_1", Atome "old_b")),
          Equiv (Atome "new_1", Non (Atome "old_a"))))
-( 20:25:36 )-< command 3 >
utop # string_of_formule tf;;
- : string = "(new_0 * ((new_0 <-> (new_1 + old_b)) * (new_1 <-> ~ old_a)))"
-( 20:27:53 )-< command 4 >
utop #
```

2. Implantation de la fonction Tseitin

Pour implanter cette fonction, j'ai mis en place trois fonctions auxiliaires :

- trois fonctions locales : ($\text{neww} : \text{int} \rightarrow \text{string}$) et ($\text{old} : \text{string} \rightarrow \text{string}$) qui permettent respectivement de générer les chaînes new_k et old_s où k et s sont respectivement un entier et une chaîne donnée en paramètre et ($\text{aux} : \text{int} \rightarrow \text{formule} \rightarrow (\text{int} * \text{formule})$) qui prend en paramètre un entier disons k et une formule disons f , et renvoie un couple contenant un entier qui représente le dernier indice utilisé pour la transformée et une formule dont sa conjonction avec l'atome new_0 donné la transformée de Tseitin de f .
- une fonction globale : ($\text{apply_op} : \text{string} \rightarrow \text{formule} \rightarrow \text{formule} \rightarrow \text{formule}$) qui permet de générer une formule binaire dont le constructeur est donné en paramètre sous forme de chaîne de caractère.

3. Implantation de la fonction restrict_inter

($\text{restrict_inter} \rightarrow \text{interpretation} \rightarrow \text{interpretation}$) : transforme une interprétation i évaluant une transformée de Tseitin comme vraie en une interprétation i' évaluant la formule de départ F comme vraie. Pour cela il suffit juste de renvoyer une fonction qui prend en paramètre un atome s et renvoie le résultat de l'interprétation de la chaîne issue de la concaténation de old_ et sur i .

4. Implantation de la fonction extention_inter

($\text{extention_inter} : \text{interpretation} \rightarrow \text{formule} \rightarrow \text{interpretation}$) : prend en paramètre une interprétation i évaluant une formule f comme vraie et renvoie une interprétation i' évaluant la transformée de Tseitin f' comme vraie. Pour cela, nous avons défini une fonction auxiliaire

($\text{to_equi_list} : \text{formule} \rightarrow \text{formule list}$) qui prend en paramètre une transformée de Tseitin f et renvoie la liste contenant que les formules d'équivalence contenues dans la formule f . Cette liste est décroissante en fonction des atomes new_k ou k est un entier.

Exemples :

soit $f = \text{Ou}(\text{Et}(\text{Et}(\text{Atome "a", Atome "b"}), \text{Atome "c"}), \text{Imp}(\text{Atome "a", Imp}(\text{Atome "b", Atome "c"})))$;

```
-( 15:55:36 )-< command 2 > { counter: 0 }-
utop # let f' = tseitin f;;
val f' : formule =
  Et
    (Atome "new_0",
     Et
       (Et
         (Equiv (Atome "new_0", Ou (Atome "new_1", Atome "new_3")),
          Et
            (Equiv (Atome "new_1", Et (Atome "new_2", Atome "old_c")),
             Equiv (Atome "new_2", Et (Atome "old_a", Atome "old_b")))),
          Et
            (Equiv (Atome "new_3", Imp (Atome "old_a", Atome "new_4")),
             Equiv (Atome "new_4", Imp (Atome "old_b", Atome "old_c")))))
      )
    )
-( 15:56:09 )-< command 3 > { counter: 0 }-
utop # to_equi_list f';
- : formule list =
[Equiv (Atome "new_4", Imp (Atome "old_b", Atome "old_c"));
Equiv (Atome "new_3", Imp (Atome "old_a", Atome "new_4"));
Equiv (Atome "new_2", Et (Atome "old_a", Atome "old_b"));
Equiv (Atome "new_1", Et (Atome "new_2", Atome "old_c"));
Equiv (Atome "new_0", Ou (Atome "new_1", Atome "new_3"))]
-( 15:56:33 )-< command 4 >
```

Description de l'algorithme utilisé:

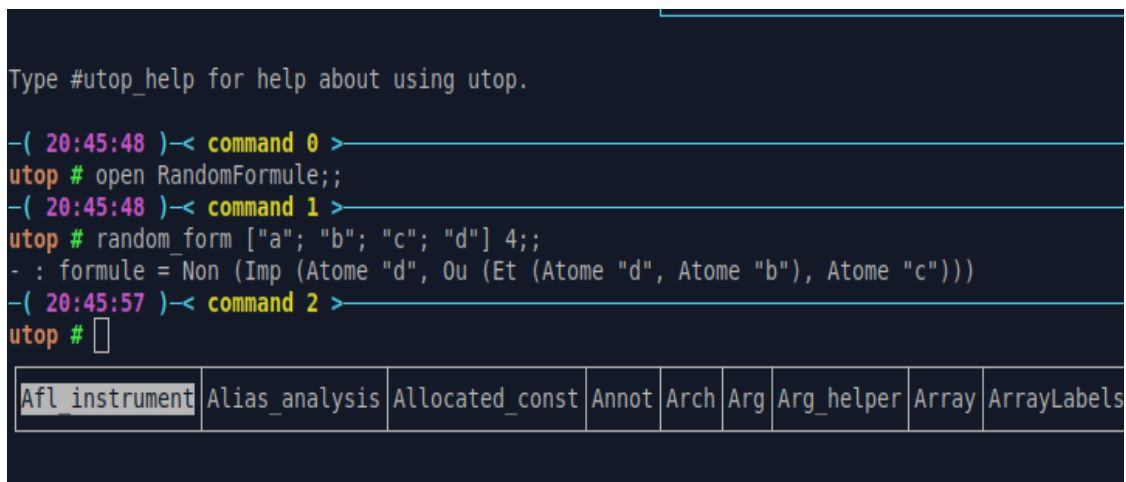
entrée : une interprétation i et une formule f

```
tf      ← transformée de tseintin de f
atomes_f ← liste des atomes de f
t       ← liste des chaînes old_s pour lesquels  $i$  s est vraie
list_equiv ← la liste des formules d'équivalence contenues dans tf
pour  $i = 0, \dots, \text{longueur}(\text{list\_equiv})$ 
     $i'$  ← interpretation_of_list t
     $s, f \leftarrow \text{list\_equiv}[i]$  où  $s = \text{new\_k}$  avec  $k$  un entier et  $f$  une formule
    si  $(\text{eval } i' f) = \text{vrai}$  alors
        ajouter  $s$  dans  $t$ 
renvoyer (interpretation_of_list t)
```

III. Génération aléatoire d'une formule

La génération aléatoire d'une formule peut se faire en utilisant une méthode récursive. Il s'agit de construire aléatoirement une formule avec un nombre donné d'opérateurs et des atomes dans un ensemble donné. Nous avons utilisé l'algorithme qui était donné dans le projet pour le mettre en place.

Exemple :



```
Type #utop_help for help about using utop.
-( 20:45:48 )-< command 0 >
utop # open RandomFormule;;
-( 20:45:48 )-< command 1 >
utop # random_form ["a"; "b"; "c"; "d"] 4;;
- : formule = Non (Imp (Atome "d", Ou (Et (Atome "d", Atome "b"), Atome "c"))))
-( 20:45:57 )-< command 2 >
utop #
```

Afl_instrument	Alias_analysis	Allocated_const	Annot	Arch	Arg	Arg_helper	Array	ArrayLabels
----------------	----------------	-----------------	-------	------	-----	------------	-------	-------------

IV. Implémentation : test empirique de validité des résultats

Dans le fichier CorrecTest.ml, nous devons implémenter les fonctions suivantes :

- test_equisat_quine** : réalisant un test empirique de l'équisatisfaisabilité des formules en utilisant l'algorithme de Quine.
Cette fonction prend en paramètre une formule f (sans occurrence de l'opérateur d'équivalence) et réalise les étapes suivantes : elle affiche f au format infixe, calcule la transformée f' de f et l'affiche au format infixe, détermine la satisfaisabilité de f selon

l'algorithme de Quine et affiche le Booléen b correspondant, détermine la satisfaisabilité de f' selon l'algorithme de Quine et affiche le Booléen b' correspondant, puis détermine le Booléen b = b' et l'affiche.

```

7
formule -> unit
8 let test_equisat_quine (f: formule) : unit =
9   let tf = (try (tseitin f) with
10    | Non_equivalence ->
11    | failwith "la formule ne doit pas contenir une opération d'équivalence"
12    )
13   in let res = "F = "^(string_of_formule f)^^"\nT(F) = "^(string_of_formule tf)^^"\n"
14   in let f_quine_sat = quine_sat f and tf_quine_sat = quine_sat tf
15   in let equisat = (f_quine_sat = tf_quine_sat)
16   in let res = res^"quine_sat(F) : "^(string_of_bool f_quine_sat)^^"\n"
17   in let res = res^"quine_sat(T(F)) : "^(string_of_bool tf_quine_sat)^^"\n"
18   in let res = res^"Equisat(T(F)) : "^(string_of_bool equisat)^^"\n"
19   in print_string res;;
20
21
string list -> int -> unit
22 let test_equisat_quine' (l : string list) (i : int) : unit =
23   let f = random_form l i
24   in test_equisat_quine f;;
25

```

- `test_equisat_dp11` : réalisant un test empirique de l'équisatisfaisabilité des formules en utilisant l'algorithme DPLL.

Cette fonction prend une formule f sans opérateur d'équivalence. Elle affiche f au format infixe. Elle calcule et affiche la transformée f' de f au format infixe. Elle détermine tous les témoins de satisfaisabilité de f avec DPLL, formant la liste ts. Elle fait de même pour f', formant la liste ts'. Elle convertit chaque témoin de ts en une interprétation satisfaisant f, créant la liste is. Elle convertit chaque témoin de ts' en une interprétation satisfaisant f', créant la liste is'. Elle convertit chaque interprétation de is pour qu'elle satisfasse f', formant is_ext. Elle convertit chaque interprétation de is' pour qu'elle satisfasse f, formant is'_res. Elle vérifie et affiche si toutes les interprétations de is_ext évaluent f comme vrai. Enfin, elle vérifie et affiche si toutes les interprétations de is'_res évaluent f comme vrai.

```

25
formule -> unit
26 let test_equisat_dp11 (f : formule) : unit =
27   let f' = (try (tseitin f) with
28    | Non_equivalence ->
29    | failwith "la formule ne doit pas contenir une opération d'équivalence"
30    )
31   in let res = "F = "^(string_of_formule f)^^"\nT(F) = "^(string_of_formule f')^^"\n"
32   in let ts = dp11_all_sat (formule_to_fcc f) and ts' = dp11_all_sat (formule_to_fcc f')
33   in let is = List.map (function x -> interpretation_of_list (all_atome_true x)) ts
34   and is' = List.map (function x -> interpretation_of_list (all_atome_true x)) ts'
35   in let is_ext = List.map (function x -> extension_inter x f) is
36   and is_res' = List.map (function x -> restrict_inter x) is'
37   in let b = is_all_eval_true f' is_ext
38   and b' = is_all_eval_true f is_res'
39   in let s = "is_ext sat T(F) ? "^(string_of_bool b)^^"\n"
40   and s' = "is'_res sat F ? "^(string_of_bool b')^^"\n"
41   in print_string (res^s);;
42
43
string list -> int -> unit
44 let test_equisat_dp11' (l : string list) (i : int) : unit =
45   let f = random_form l i
46   in test_equisat_dp11 f;;
47
(*

```

- `test_equisat_quine'` et `test_equisat_dp11'` : utilisant le module `RanfonFormule` afin de générer une formule aléatoire et appliquant les tests précédents.

V. Évaluation des performances

Le fichier `Benchmark.ml` contient les fonctions suivantes :

- `bench_dp11` : réalisant une évaluation des performances de l'algorithme DPLL sur une formule donnée.
Celle prend en paramètre une formule `f` (sans occurrence de l'opérateur d'équivalence) et réalise les étapes suivantes : elle affiche `f` au format infixe, calcule la transformée `f'` de `f`, mesure le temps de calcul et affiche ces informations. Ensuite, elle calcule la Forme Clausale Conjonctive (FCC) `fcc` associée à `f`, mesure le temps de calcul et affiche ce temps ainsi que le nombre de clauses et de littéraux. De plus, elle calcule la FCC associée à `f'`, mesure le temps de calcul et affiche ce temps ainsi que le nombre de clauses et de littéraux. Enfin, elle détermine la satisfaisabilité de `fcc` avec `dp11_sat_unit_prop`, mesure le temps de calcul et affiche ces informations, puis détermine la satisfaisabilité de `f`.
- `bench_dp11'` : générant une formule aléatoire et évaluant les performances de l'algorithme DPLL. Cette fonction utilise le module `RandomFormule` pour avoir aléatoirement une fonction et appelle la fonction `bench_dp11'` en lui donnant la formule comme paramètre.

```

15 formule -> unit
16 let bench_dp11 (f : formule) : unit =
17   let (f', t) = (try (mesure tseitin f) with
18     | Non_equivalence ->
19       failwith "la formule ne doit pas contenir une opération d'équivalence"
20     )
21   in let res = "F = "^(string_of_formule f)^\nT(F) = "^(string_of_formule f')^\n"
22   in let res = res^"transformation = "^(string_of_float t)^\n"
23   in let fcc, t = mesure formule_to_fcc f
24   in let nb_clause, nb_lit = nb_clause_lit fcc
25   in let res = res^"fcc (F) : "^(string_of_float t)^\n ("^(string_of_int nb_clause)^
26     " clauses, "^(string_of_int nb_lit)^\n littéraux)\n"
27   in let fcc', t = mesure formule_to_fcc f'
28   in let nb_clause, nb_lit = nb_clause_lit fcc'
29   in let res = res^"fcc' (T(F)) : "^(string_of_float t)^\n ("^(string_of_int nb_clause)^
30     " clauses, "^(string_of_int nb_lit)^\n littéraux)\n"
31   in let b1, t = mesure dp11_sat_unit_prop fcc
32   in let res = res^"dp11 (F) : "^(string_of_bool b1)^\n ("^(string_of_float t)^\n"
33   in let b2, t = mesure dp11_sat_unit_prop fcc'
34   in let res = res^"dp11 (T(F)) : "^(string_of_bool b2)^\n ("^(string_of_float t)^\n"
35   in print_string res;;
36
37
38 string list -> int -> unit
39 let bench_dp11' (l : string list) (i : int) : unit =
40   bench_dp11 (random_form l i);
41

```

VI. Conclusion

En résumé, ce projet vise à implémenter l'algorithme de transformation de Tseitin en OCaml et à tester empiriquement ses performances en utilisant les formules aléatoires et des algorithmes de résolution de satisfaisabilité. Le but est d'approfondir la compréhension des processus

fondamentaux de cette transformation et d'évaluer son efficacité lorsqu'elle est appliquée concrètement.